~~Zero~~ *Low*-Overhead Dispatch

Leveraging __musttail and custom ABI to write a fast interpreter in C++

Paolo Severini

C++ DAY 2026

DrumBrake: a WebAssembly interpreter for V8

A production interpreter

- Development started around 2020; used in Microsoft Edge, on tvOS and in GrapheneOS's Vanadium browser
- Works on Windows, Linux, MacOS on x64 and ARM64
- Supports modern Wasm features: Memory64, WasmGC, SIMD, threads and atomics, …
- Integrated with V8's JavaScript execution environment

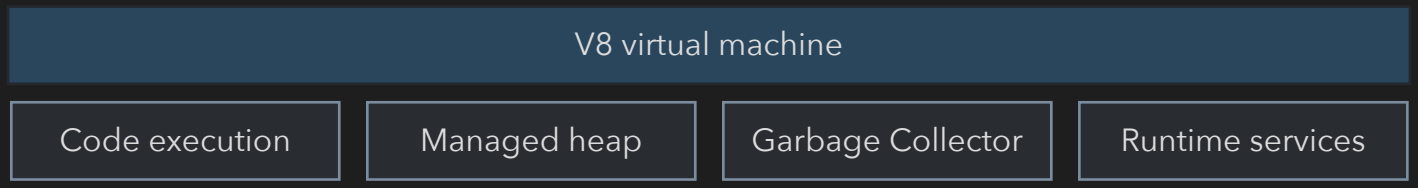
Inspired by **Wasm3**

- A fast WebAssembly interpreter and runtime written in C
- Its "meta-machine" chains instruction handlers through tail calls, carrying interpreter state as function arguments that map to CPU registers.

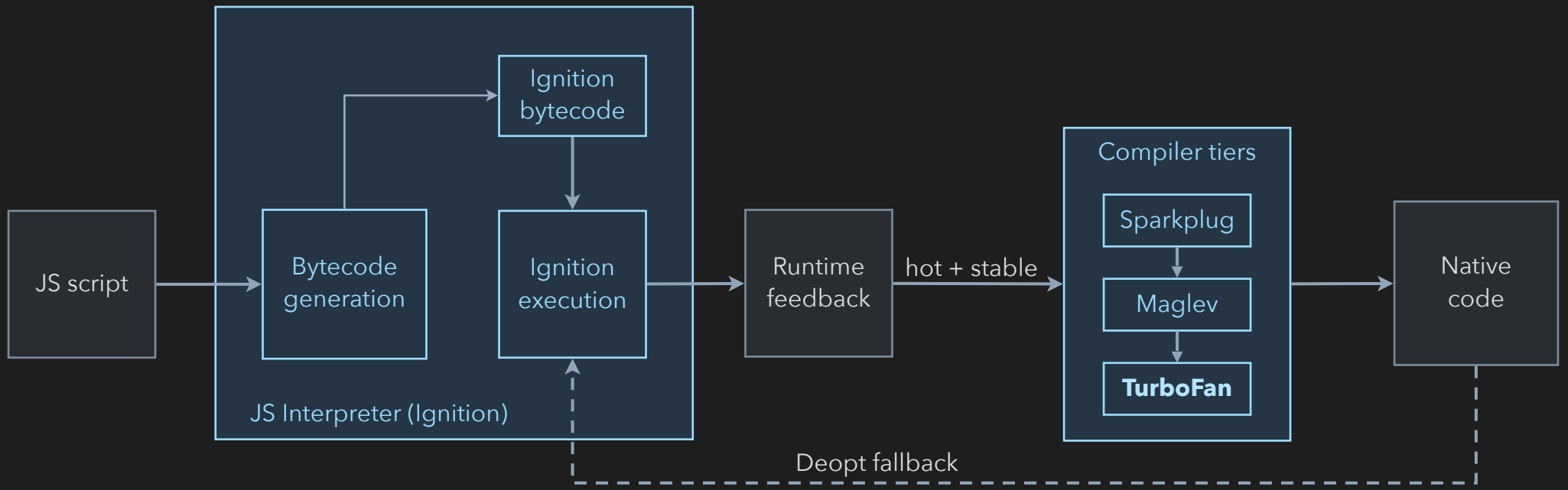
What is V8?

V8: a JavaScript and WebAssembly engine

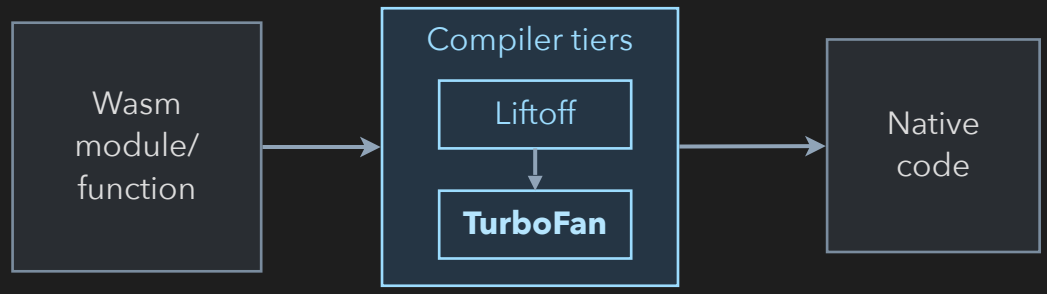
- Open-source execution engine written in C++
- Used in Chrome, Chromium-based browsers such as Edge, and Node.js
- Executes JavaScript and WebAssembly, with runtime services such as memory management and garbage collection.
- Embeddable in C++ applications—not limited to browsers



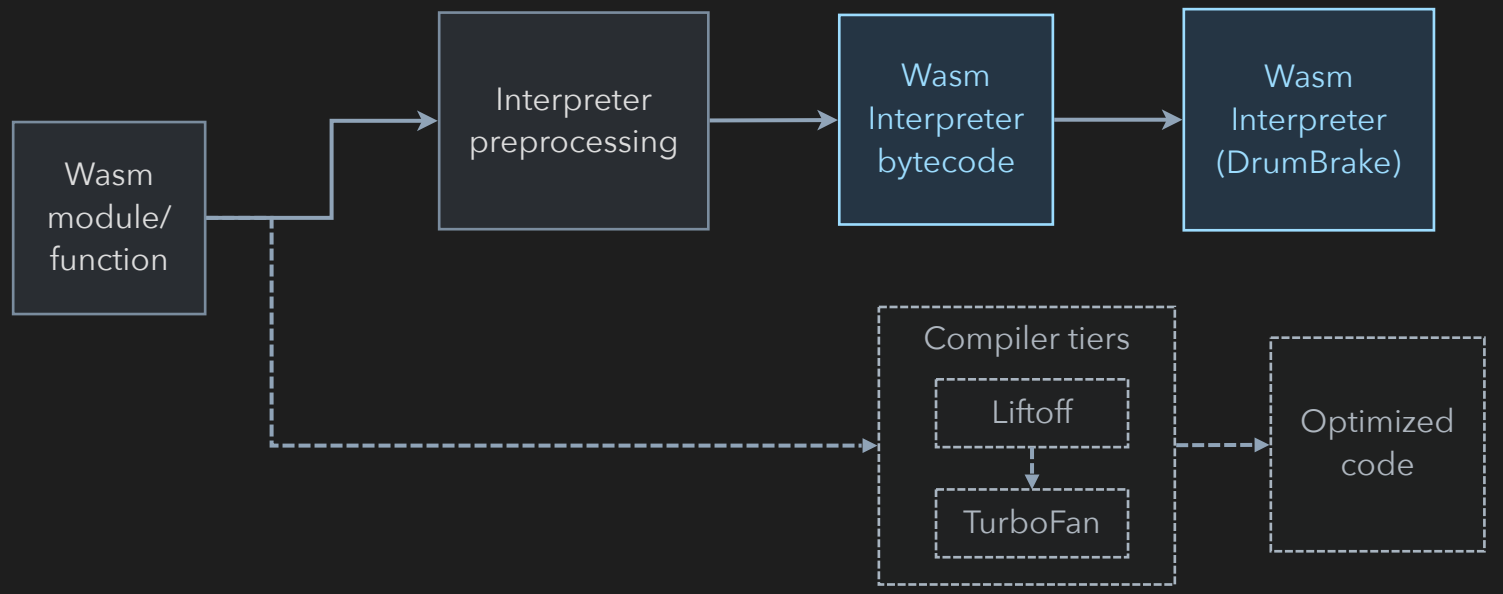
V8 - JavaScript pipeline



V8 - WebAssembly pipeline



WebAssembly without a JIT



Jitless - no runtime generation of native code for the Wasm program

- Restricted execution environments: native-code generation may be unavailable or prohibited
- Reduced attack surface: remove the JIT compilation path
- The purpose is not to beat Liftoff at getting started

The engineering challenge is to make that interpreter **fast** enough to be useful

Interpreter: large switch loop

```
void ExecuteAdd() {...}
void ExecuteLoad() {...}

while(running) {
  switch (ReadOpcode()) {
    case ADD:
      ExecuteAdd();
      break;
    case LOAD:
      ExecuteLoad();
      break;
    ...
  }
}
```

Instruction selection and execution within one loop

The compiler determines how the switch is dispatched

Interpreter: central loop + handlers

```
// Inside class Interpreter:

using Handler = void (Interpreter::*)();

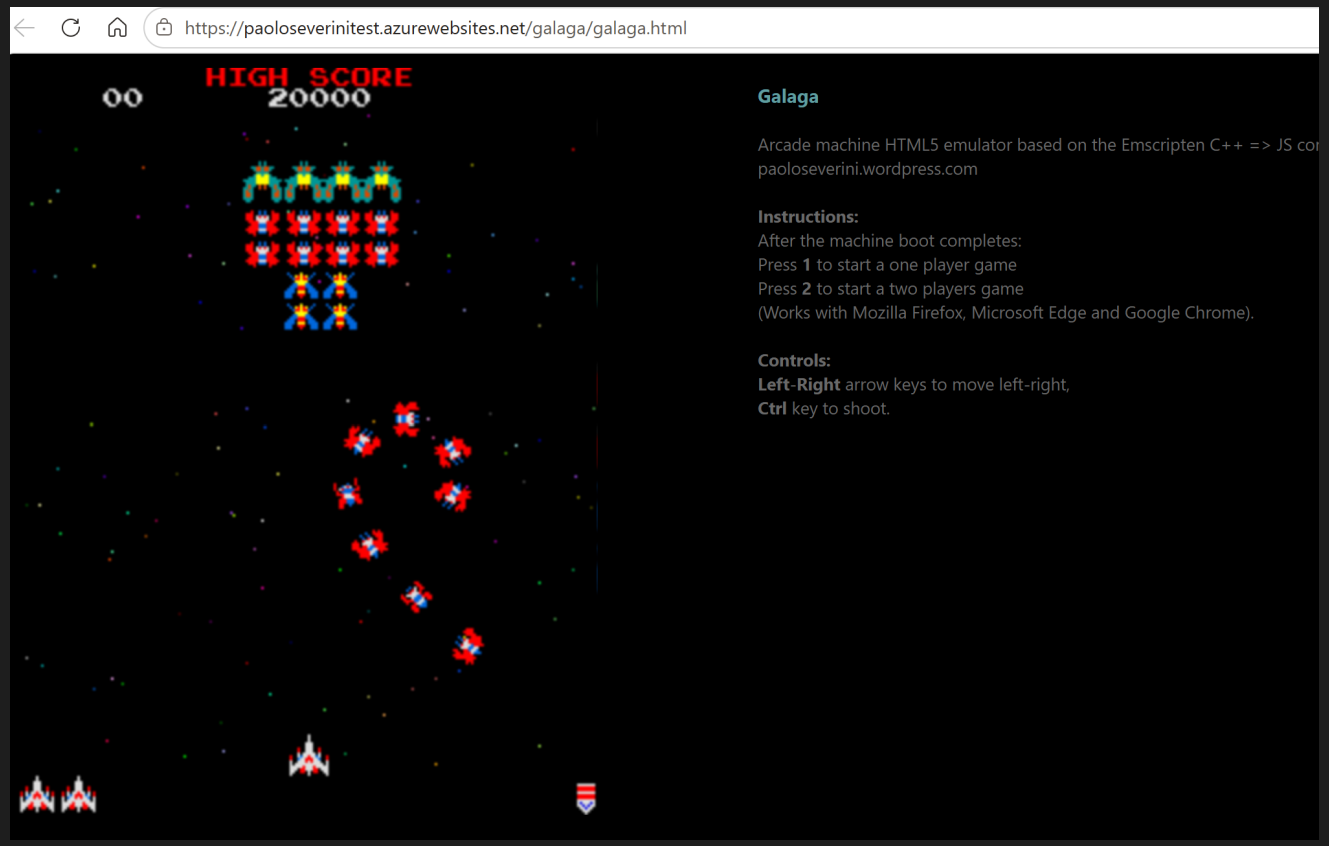
void ExecuteAdd() {...}

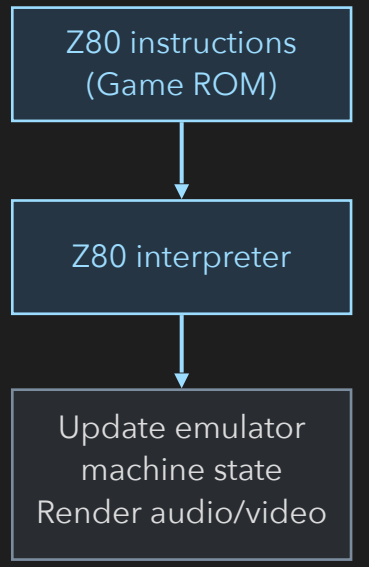
void Run() {
  while (running) {
    Handler handler = FetchDecode(pc);
    (this->*handler)();
  }
}
```

Separate instruction handlers, selected through a dispatch table

Each handler returns to the central loop

Example: Galaga emulator





Z80 interpreter written in C++

https://paoloseverini.wordpress.com/2016/02/13/galaga-an-arcade-machine-emulator-for-windows-and-html5/

Example: Galaga emulator

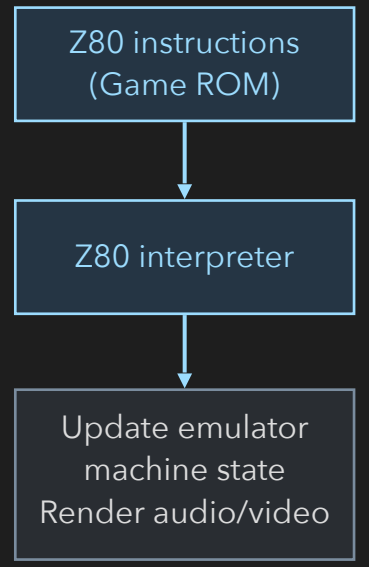
```
// Inside class Z80ProcessorImpl:

void Adc_a_n()
{
  byte result = static_cast<byte>((this->A + this->c + this->instr.immediate) & 0xff);
  this->F = Get_SZHVC_add_flag(this->A, result, this->c);
  this->A = result;
}

OpcodeInfo fetchDecodeTable[256] = {
  OpcodeInfo(DecodeAction::None, &Z80ProcessorImpl::Nop), // 0x00
  OpcodeInfo(DecodeAction::ImmediateExtended, &Z80ProcessorImpl::Ld_bc_nn), // 0x01
  ...
  OpcodeInfo(DecodeAction::Immediate, &Z80ProcessorImpl::Adc_a_n), // 0xce
  ...
}

void FetchDecode(ushort pc)
{
  byte opcode = ReadMemory(pc++);
  // Fill this->instr with opcode, operands...
  ...
}

// Main loop (interrupt handling omitted)
do {
  FetchDecode(this->PC);
  this->PC += static_cast<ushort>(this->instr.length);
  this.R++;
  Action execInstruction = this->instr.execute;
  ((*this).*execInstruction)();
  cyclesLeft -= this->instr.cycles;
}
while (!halt && cyclesLeft > 0);
```



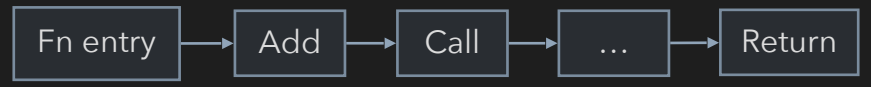
Tail-call-threaded interpreter

```
void handler(State* state) {
  execute(state);

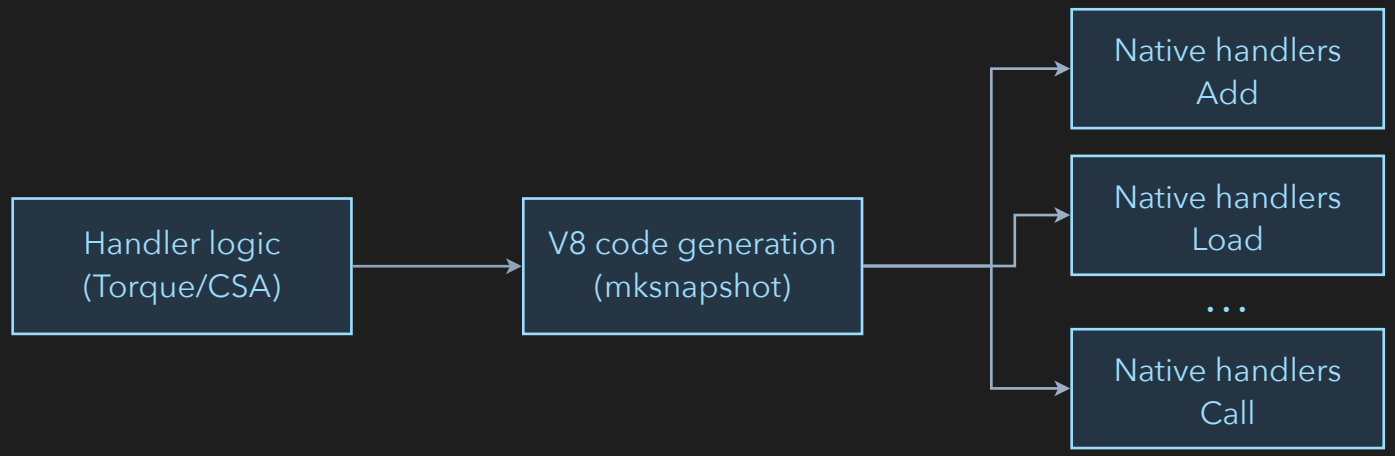
  auto id = read_next_id(state);
  auto next = handlers_[id];

  [[clang::musttail]]
  return next(state);
}
```

Tail-call chain of handlers: each handler transfer control to the next



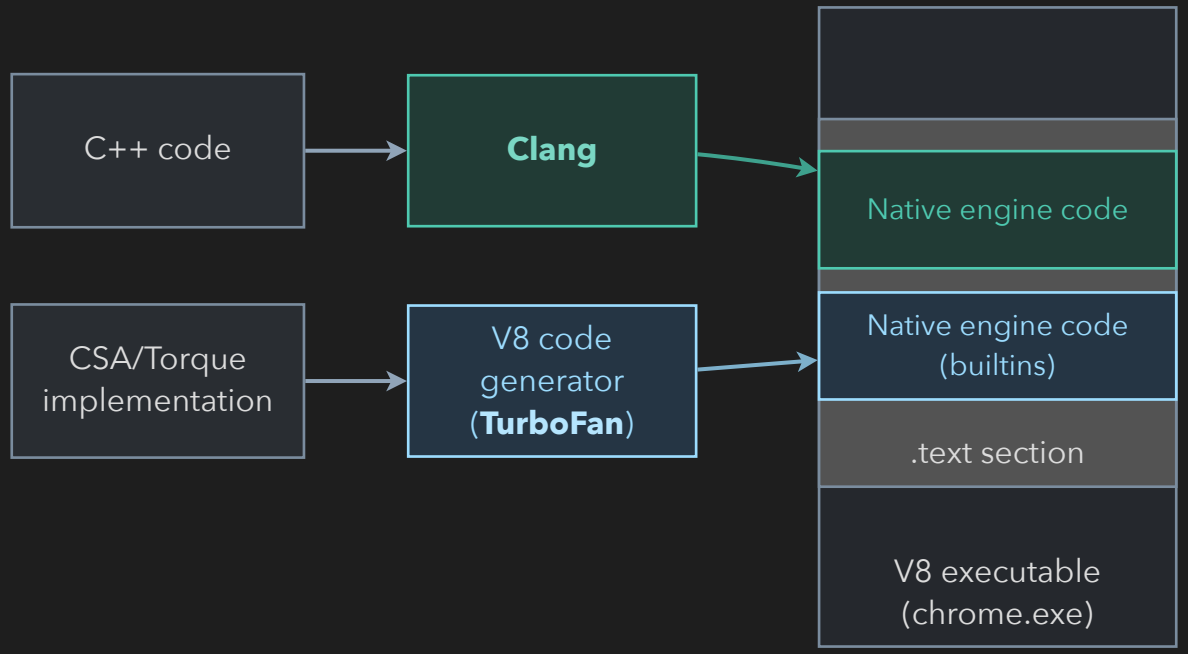
Ignition: instruction handlers with TurboFan



The V8 JS interpreter (Ignition) uses the chain of handlers approach, pre-compiling (at engine build time) the instruction handlers using the TurboFan compilers

V8 can generate its own engine code

Build-time pipeline

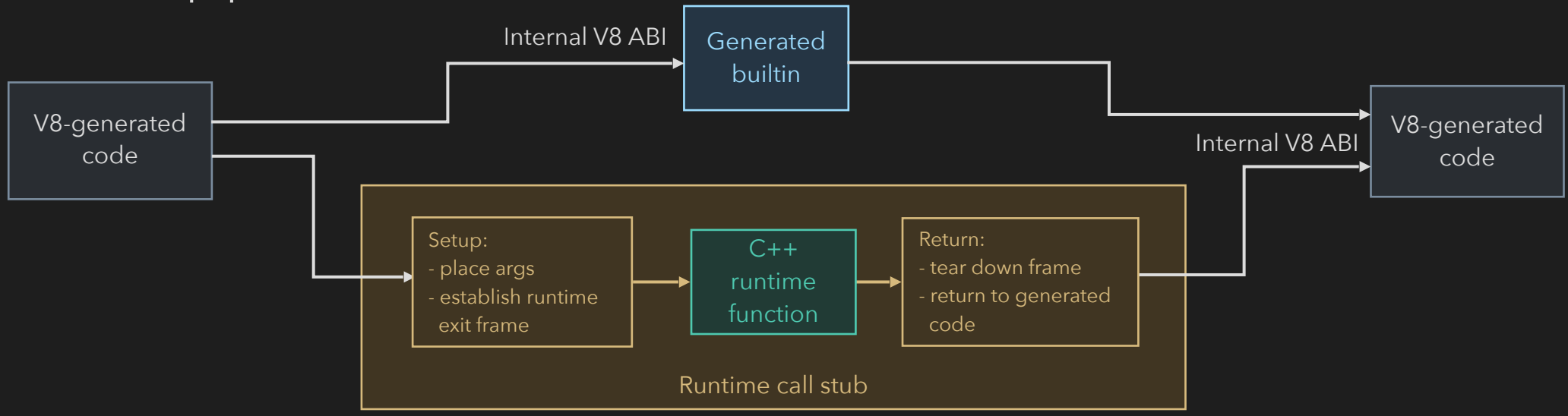


The executable contains machine code generated with different generators and calling conventions:

- Implementation code compiled with Clang
- V8 builtins precompiled with TurboFan (with their own V8-specific ABI)

Crossing into the C++ runtime has a cost

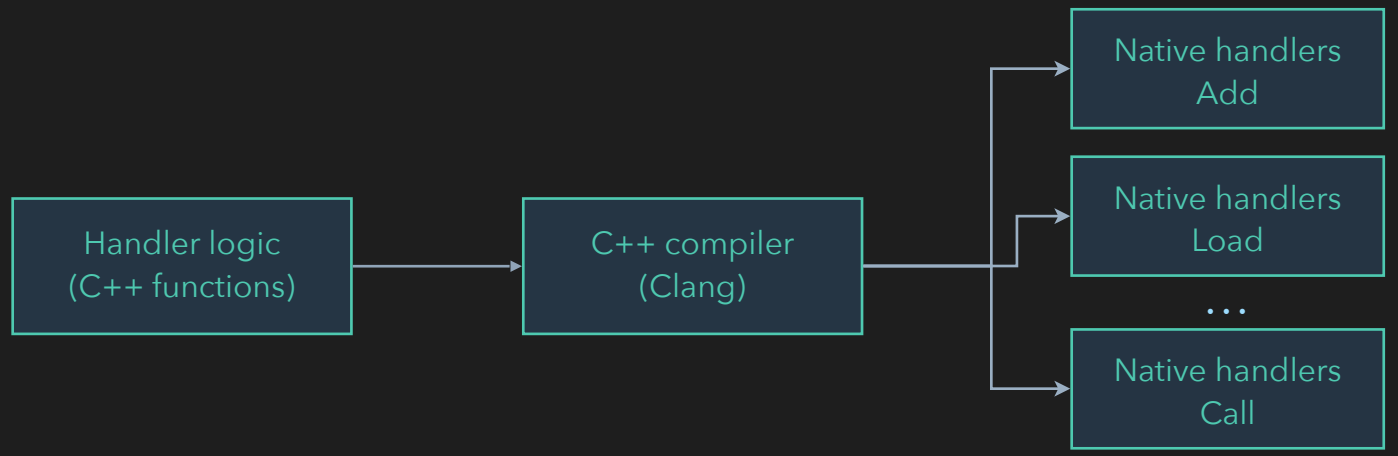
Run-time pipeline



Where the code runs, and how we enter it, matters:

- V8-generated code uses its own (platform-)specific ABI
- A small operation can become expensive if reaching it requires a runtime transition (stub)

DrumBrake: instruction handlers in C++



For the Wasm interpreter, we also use a chain of handlers, but with handlers implemented as C++ functions and compiled with Clang

Wasm bytecode as the input representation

Original C++ code

```
uint32_t example(uint32_t a,
                 uint32_t b,
                 uint32_t c) {
  return (a + b) * c;
}
```



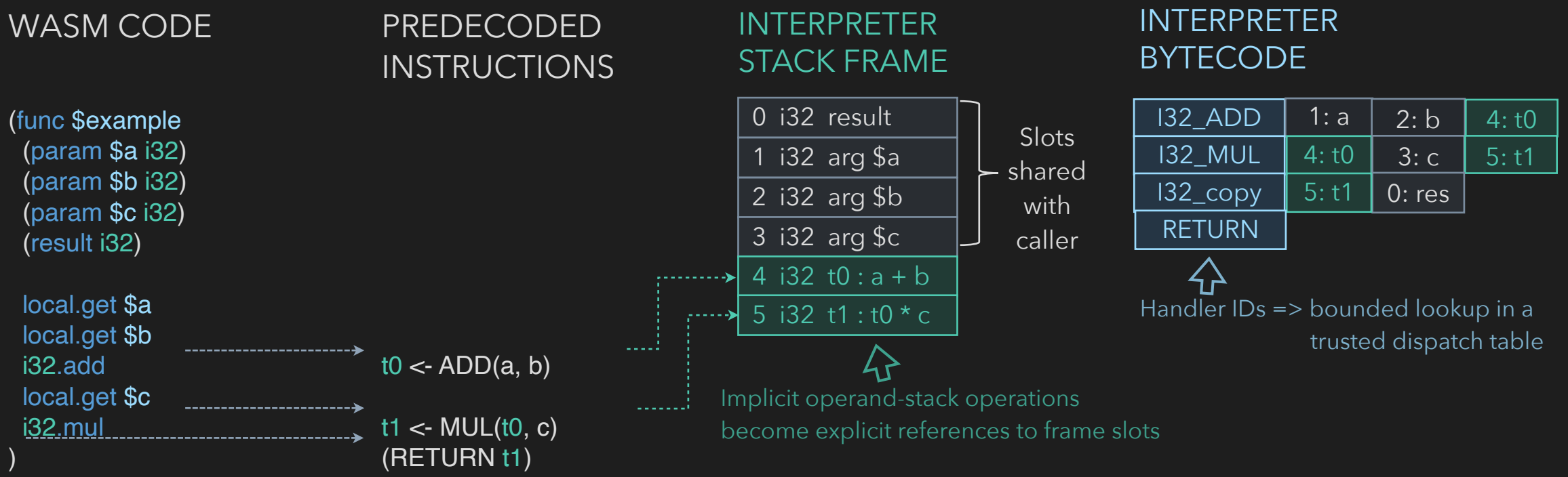
Equivalent Wasm, illustrative

```
(func $example
  (param $a i32) (param $b i32) (param $c i32)
  (result i32)

  local.get $a          [ a ]
  local.get $b          [ a, b ]
  i32.add               [ t0=a+b ]
  local.get $c          [ t0, c ]
  i32.mul               [ t1=t0*c ]
)                       Operand stack
```

WebAssembly is a virtual stack machine that computes by pushing and popping operands on an execution stack.

Interpreter bytecode and fixed frame slots



Before executing a function, DrumBrake preprocesses its Wasm bytecode:

- Generates internal interpreter **bytecode** for the instruction handlers to execute
- Computes a fixed **frame layout**, assigning slots to locals and intermediate values, and encoding where each instruction reads its operands and writes its results.
- **local.get** instructions can identify existing frame slots (and be omitted in the bytecode)

Function calls and interpreter frames

C++ code

```
int bar(int v) {
  ...
}

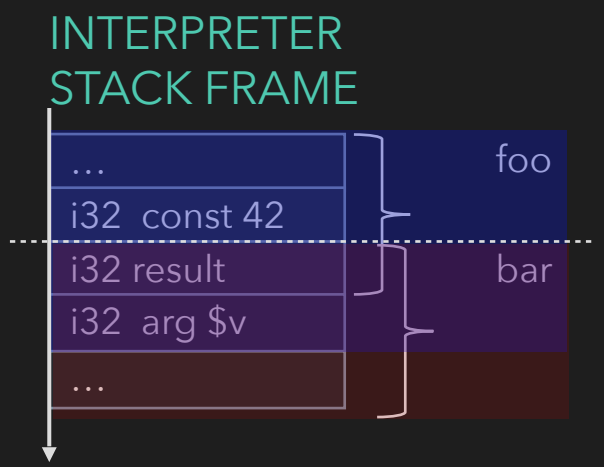
int foo() {
  return bar(42) + 1;
}
```



Wasm bytecode

```
(func $bar
  (param $v i32) (result i32)
  ...
)

(func $foo (result i32)
  i32.const 42
  call $bar
  ...
)
```



Separate call frames, shared argument and result slots

- An ordinary Wasm call adds a native call frame
- In the interpreter stack, the caller's outgoing argument slots become the callee's parameter slots
- The callee writes its result into a slot the caller can read after the return

Instruction handler: decode, execute, dispatch

```
void Add(State* state)
{
  // Decode
  int lhs = ReadSlotIndex(state);
  int rhs = ReadSlotIndex(state);
  int dst = ReadSlotIndex(state);

  // Execute
  state->slots[dst] =
      state->slots[lhs] + state->slots[rhs];

  // Dispatch
  auto id = ReadHandlerId(state);
  auto next = handlers[id];
  next(state);
}
```

Decode: read the operands from interpreter bytecode

Execute: perform the operation on frame slots.
Each handler reads its operands from frame slots and writes its result to a slot

Dispatch: invoke the next handler
The bytecode contains handler IDs, not handler addresses, for additional safety

Guaranteed tail calls as a compiler contract

```
[[clang::musttail]] return next(state);
```

A required tail-call – not an optimization hint
Handler-to-handler dispatch does not accumulate native call frames

Supported by Clang, GCC (since 15.1, April 2025) and MSVC (since Build Tools 14.50, still experimental)

Guaranteed tail calls - constraints

Three [[clang::musttail]] constraints:

- Matching signatures and calling conventions (use same prototype for all handlers)
- No pending nontrivial destruction (finish RAII scopes before dispatch)
- Caller-local lifetimes end before transfer (do not hand the next handler references to local objects)

```
void handler(State* state)
{
  {
    Guard guard(state);
    execute(state);
  }

  [[clang::musttail]] return next(state);
}
```

Guard has a non-trivial destructor

next must have the same signature

Anatomy of an instruction handler

A production handler (macro parameters expanded for readability):

```
__attribute__((noinline)) static DISABLE_CFI_ICALL
void __vectorcall s2s_I32Add(const uint8_t* code, uint32_t* sp,
                             WasmInterpreterRuntime* wasm_runtime,
                             int64_t r0, double fp0) {
  uint32_t rval = pop<uint32_t>(sp, code, wasm_runtime);
  uint32_t lval = pop<uint32_t>(sp, code, wasm_runtime);

  push<uint32_t>(sp, code, wasm_runtime, (lval + rval));

  auto next = kInstructionTable[ReadFnId(code) & kInstructionTableMask];
  [[clang::musttail]] return next(code, sp, wasm_runtime, r0, fp0);
}
```

Decode: pop<T> reads operands from precomputed frame slots

Execute: Compute the result; push<T> places it in a frame slot

Tail-dispatch: Select the next handler and pass the state forward

What the compiler emits – x64 Windows

s2s_I32Add

```
;Read operand-slot offsets
movzx eax, word ptr [rcx]            ;rhs slot
movzx r10d, word ptr [rcx+2]         ;lhs slot

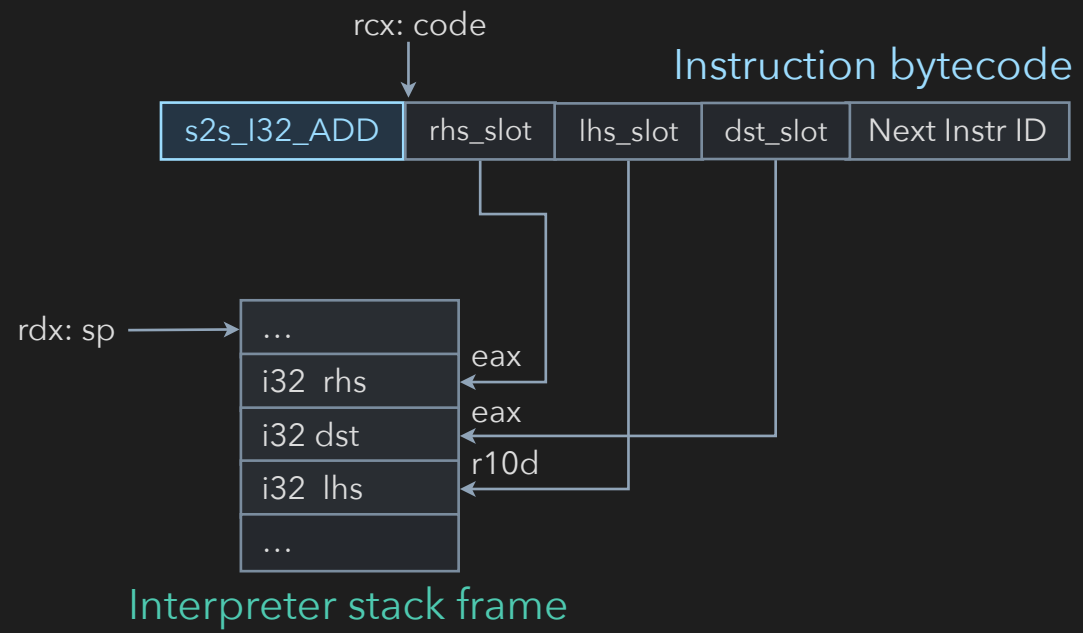
;Read values from slots and compute result in r10
mov   r10d, dword ptr [rdx+r10*4]
add   r10d, dword ptr [rdx+rax*4]

;Decode destination slot and store the result
movzx eax, word ptr [rcx+4]          ;dst slot
mov   dword ptr [rdx+rax*4], r10d

;Read next handler ID and advance code
movzx eax, word ptr [rcx+6]
add   rcx, 8

;Bound the index and tail-dispatch
and   eax, 0FFFh
lea   r10, [rip+kInstructionTable]
jmp   qword ptr [r10+rax*8]
```

code	sp	runtime	r0	fp0
RCX	RDX	R8	R9	xmm4



What the compiler emits – macOS ARM64

s2s_I32Add

```
; Decode the rhs slot offset and read its value
ldrh   w8, [x0]             ; rhs slot offset
ldr    w8, [x1, x8, lsl #2]  ; rhs = frame[rhs_slot]

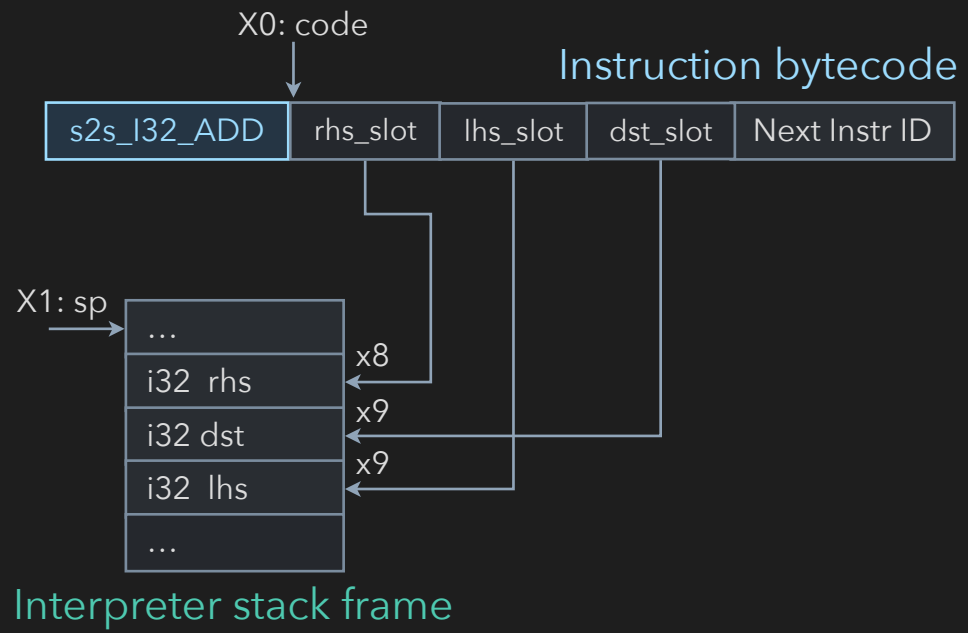
; Decode the lhs slot offset and read its value
ldrh   w9, [x0, #0x2]        ; lhs slot offset
ldr    w9, [x1, x9, lsl #2]  ; lhs = frame[lhs_slot]

; Compute the sum and store it in the destination slot
add    w8, w9, w8
ldrh   w9, [x0,#0x4]         ; dst offset
str    w8, [x1, x9, lsl #2]  ; frame[dst_slot] = lhs + rhs

; Resolve the next handler
ldrh   w8, [x0, #0x6]        ; next handler ID
and    x8, x8, #0xfff        ; mask the handler-table index
adrp   x9, PAGE_ADDR         ; page containing kInstructionTable
add    x9, x9, #0x3b0        ; complete the table's address
ldr    x4, [x9, x8, lsl #3]  ; next = table[id]: 8-byte pointers

; Advance code and tail-dispatch
add    x0, x0, #0x8          ; advance past three offsets and ID
br     x4                    ; tail-jump to the next handler
```

code	sp	runtime	r0	fp0
x0	x1	x2	x3	d0



How do we pass arguments in registers?




```
void handler(const uint8_t* code, uint32_t* sp,
             WasmInterpreterRuntime* runtime,
             int64_t r0, double fp0);
```

One handler's result becomes the next handler's argument

The calling convention (ABI) determines which arguments arrive in registers and which are pushed on the stack

Calling conventions

```
void HANDLER_ABI handler(const uint8_t* code, uint32_t* sp,
                         WasmInterpreterRuntime* runtime, int64_t r0, double fp0);
```

x64 Windows	code	sp	runtime	r0	fp0
Default ABI	RCX	RDX	R8	R9	stack
__vectorcall	RCX	RDX	R8	R9	xmm4

arm64 Windows	code	sp	runtime	r0	fp0
Default ABI	x0	x1	x2	x3	d0

x64 Linux/ MacOS					
Default ABI	RDI	RSI	RDX	RCX	xmm0
__vectorcall	RCX	RDX	R8	R9	xmm4

arm64 Linux/ MacOS					
Default ABI	x0	x1	x2	x3	d0

- **Windows x64**
  - The default ABI passes code, sp, runtime, and r0 in registers, but passes fp0 on the stack
  - __vectorcall allows us to pass all five arguments in registers
- **System V x64 and ARM64**
  - The default ABI already passes all five arguments in registers

The modified handler

The same handler, optimized to pass the result in a register to the next handler (x64):

```
__attribute__((noinline)) static DISABLE_CFI_ICALL
void __vectorcall s2r_I32Add(const uint8_t* code, uint32_t* sp,
                             WasmInterpreterRuntime* wasm_runtime,
                             int64_t r0, double fp0) {
  uint32_t rval = pop<uint32_t>(sp, code, wasm_runtime);
  uint32_t lval = pop<uint32_t>(sp, code, wasm_runtime);

  r0 = lval + rval;

  auto next = kInstructionTable[ReadFnId(code) & kInstructionTableMask];
  [[clang::musttail]] return next(code, sp, wasm_runtime, r0, fp0);
}
```

Handler variants encode where values come from and where they go:

s2s: slot operands => slot result
s2r: slot operands => register result
r2r: register operand [+ slot operand] => register result
r2s: register operand [+ slot operand] => slot result

Intermediate values carried in a register

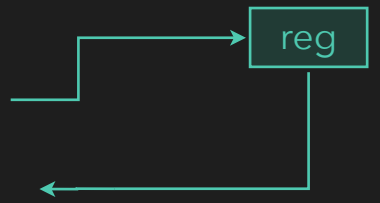
WASM CODE

```
(func $example
  (param $a i32)
  (param $b i32)
  (param $c i32)
  (result i32)

  local.get $a
  local.get $b
  i32.add
  local.get $c
  i32.mul
)
```

PREDECODED INSTRUCTIONS

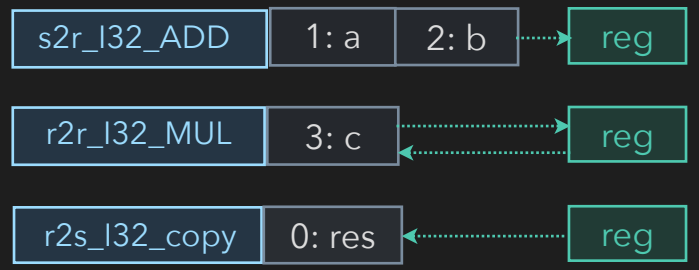
```
reg <- ADD(a, b)
reg <- MUL(reg, c)
```



INTERPRETER STACK FRAME



INTERPRETER BYTECODE



Carry intermediate results in a register through the handler chain

The result of a handler becomes an argument to the next handler

Register arguments – x64 Windows

s2r_I32Add

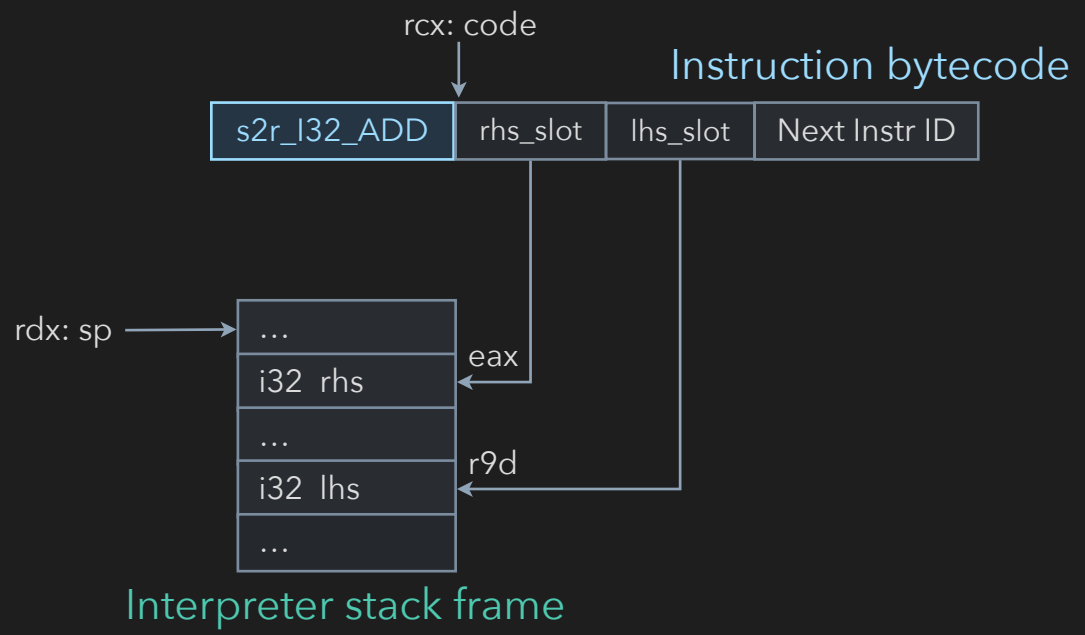
```
;Read operand-slot offsets
movzx eax, word ptr [rcx]
movzx r9d, word ptr [rcx+2]

;Read values from slots and compute result in r0
mov   r9d, dword ptr [rdx+r9*4]
add   r9d, dword ptr [rdx+rax*4]

;Read next handler ID and advance code
movzx eax, word ptr [rcx+4]
add   rcx, 6

;Bound the index and tail-dispatch
and   eax, 0FFFh
lea   r10, [rip+kInstructionTable]
jmp   qword ptr [r10+rax*8]
```

code	sp	runtime	r0	fp0
RCX	RDX	R8	R9	xmm4



The cost of interpretation

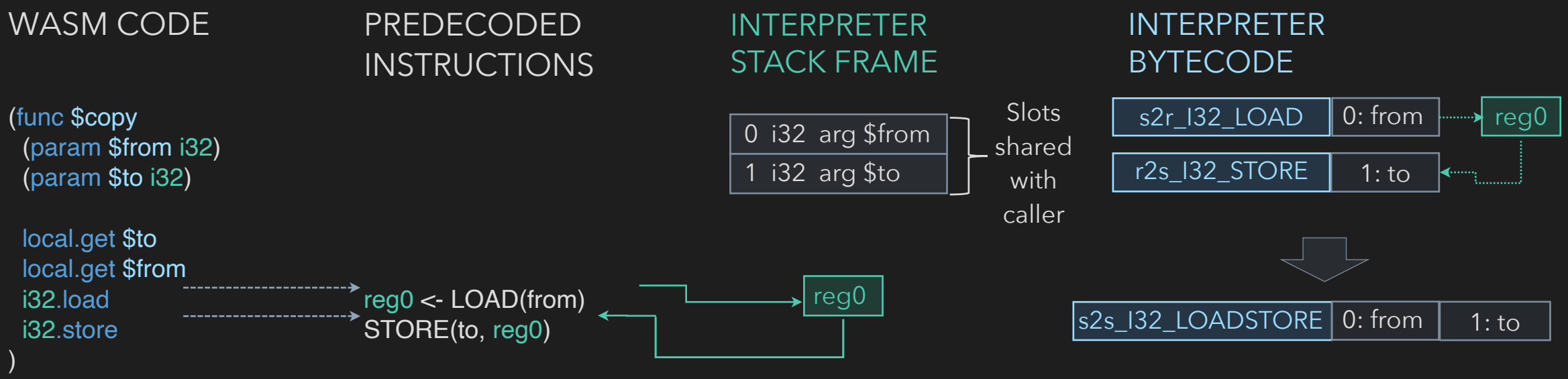
Workload	DrumBrake	TurboFan	Execution-time ratio
HashSet	220.0 ms	17.1 ms	12.9x
Quicksort	141.4 ms	19.0 ms	7.4x
TSF	197.7 ms	9.4 ms	21.0x
Argon2	641.1 ms	20.3 ms	31.6x
zlib	274.9 ms	31.6 ms	8.8x
SQLLite	733.5 ms	26.1 ms	28.1x
Galaga (5000 steps)	41867.8 ms	2226.9 ms	18.8x

Configuration: Intel Core Ultra 9 285K, Windows 11, V8 15.3.1

“Fast enough” to run large Wasm application like Google Earth, but not for very CPU-intensive workloads

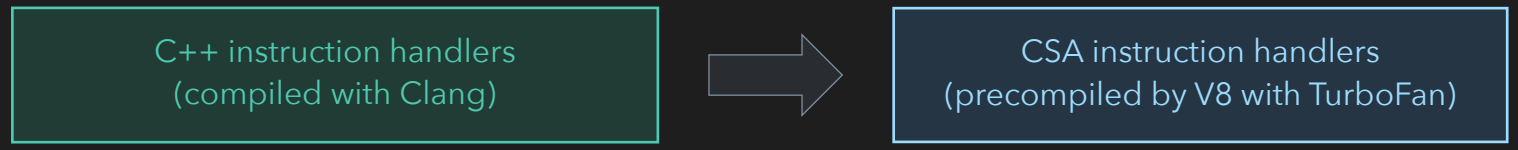
Fused instructions eliminate dispatches

```
void copy(const uint32_t* from, uint32_t* to) {
  *to = *from;
}
```



Some common instruction pairs (like Load + Store) can be **fused**, to reduce the number of dispatches

The CSA experiment



Ignition, V8's JS interpreter, uses the TurboFan-precompiled builtins approach

Hypothesis: CSA (Code Stub Assembler) handlers with a custom register convention could outperform Clang-generated C++ handlers

Results:

Benchmark	C++ handlers	CSA handlers	Result
Galaga	4885 msec	5017 msec	**+2.7%**

The CSA experiment

Benchmark	C++ handlers	CSA handlers	Result
Galaga	4885 msec	5017 msec	**+2.7%**

- Why did CSA lose on Galaga?
  - CSA provides control over a custom register-based ABI
  - Clang already optimized the C++ handler chain very effectively
  - CSA introduced extra state loads, moves and conversions (ex.: ~9 instructions with Clang vs ~14 with TurboFan)
  - Complex paths implemented through CallRuntime incurred runtime-transition overhead
- Conclusion: Keep Clang's code generation while carrying more interpreter state in registers

preserve_none: changing the register contract

```
__attribute__((preserve_none)) void handler(/* interpreter state */);
```

preserve_none: an experimental calling convention that treats most general-purpose registers as caller-saved and **uses more of them to pass function arguments**, reducing the callee's register-preservation obligations

- **x64**: only RSP and RBP remain preserved
- **arm64**: preserve x29 (frame pointer), x30 (link register), maintain SP
- Floating-point register rules are unchanged

The callee may overwrite those registers without restoring them

The caller must protect the values it needs after the call

The preserve_none experiment

An internal ABI tailored to the handler chain:

```
void __attribute__((preserve_none)) handler(const uint8_t *code, uint32_t *sp,
                                            WasmInterpreterRuntime *wasm_runtime,
                                            int64_t r0, double fp0,
                                            uint8_t *memory0_start, void *instruction_table,
                                            int64_t r1, double fp1);
```

Carry more interpreter state in registers between handlers:

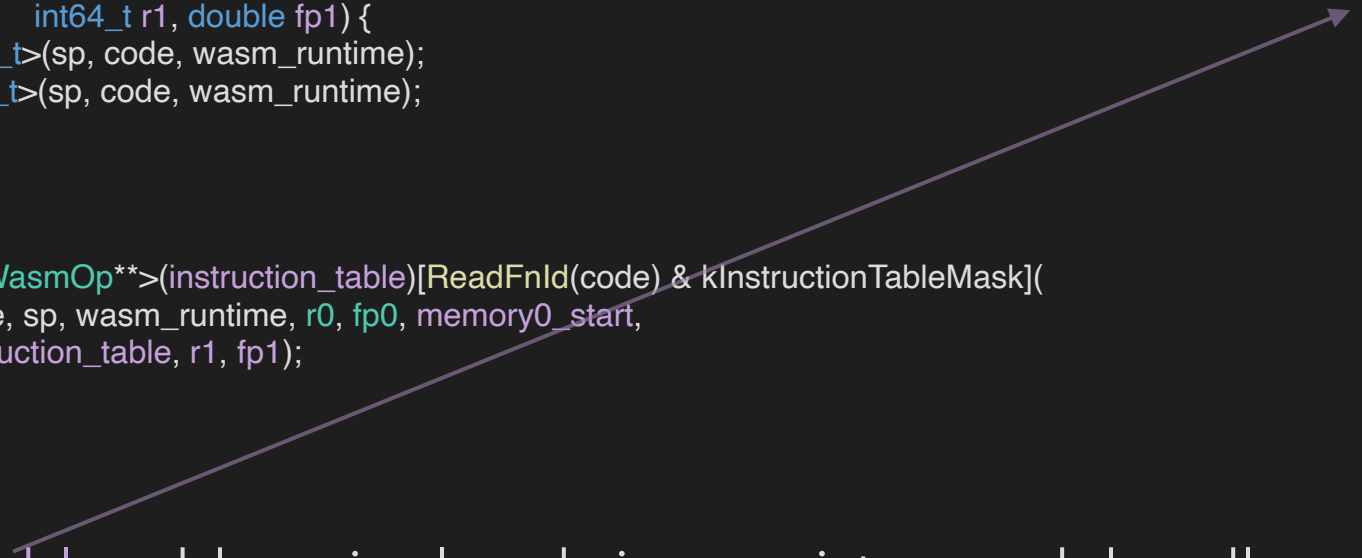
- Additional integer and floating-point registers (r1 , fp1) provide room to retain more intermediate values between handlers
- Dispatch-table pointer: avoid reconstructing its address in each handler
- Linear-memory base pointer: make (first) memory's base directly available to load/store handlers

A preserve_none instruction handler

```
__attribute__((noinline)) static DISABLE_CFI_ICALL
void __attribute__((preserve_none)) s2r1_I32Add(const uint8_t *code, uint32_t *sp,
                                    WasmInterpreterRuntime *wasm_runtime,
                                    int64_t r0, double fp0,
                                    uint8_t *memory0_start, void *instruction_table,
                                    int64_t r1, double fp1) {
  uint32_t rval = pop<uint32_t>(sp, code, wasm_runtime);
  uint32_t lval = pop<uint32_t>(sp, code, wasm_runtime);

  r1 = lval + rval;

  [[clang::musttail]]
  return reinterpret_cast<PWasmOp**>(instruction_table)[ReadFnId(code) & kInstructionTableMask](
                            code, sp, wasm_runtime, r0, fp0, memory0_start,
                            instruction_table, r1, fp1);
}
```



The dispatch-table address is already in a register: each handler uses the forwarded pointer instead of reconstructing the table's address

A preserve_none instruction handler

```
__attribute__((noinline)) static DISABLE_CFI_ICALL
void __attribute__((preserve_none)) s2r1_I32Add(const uint8_t *code, uint32_t *sp,
                                    WasmInterpreterRuntime *wasm_runtime,
                                    int64_t r0, double fp0,
                                    uint8_t *memory0_start, void *instruction_table,
                                    int64_t r1, double fp1) {
  uint32_t rval = pop<uint32_t>(sp, code, wasm_runtime);
  uint32_t lval = pop<uint32_t>(sp, code, wasm_runtime);

  r1 = lval + rval;

  [[clang::musttail]]
  return reinterpret_cast<PWasmOp**>(instruction_table)[ReadFnId(code) & kInstructionTableMask](
                            code, sp, wasm_runtime, r0, fp0, memory0_start,
                            instruction_table, r1, fp1);
}
```

A second integer cache keeps another intermediate result in a register:

- This handler computes into r1 while preserving the value in r0
- A subsequent handler can consume both without reloading them from the interpreter frame (=> saving mov instructions)

preserve_none – x64

code	sp	runtime	r0	r1	fp0	fp1	mem base	disp. table
R12	R13	R14	R15	RDX	xmm0	xmm1	RDI	RSI

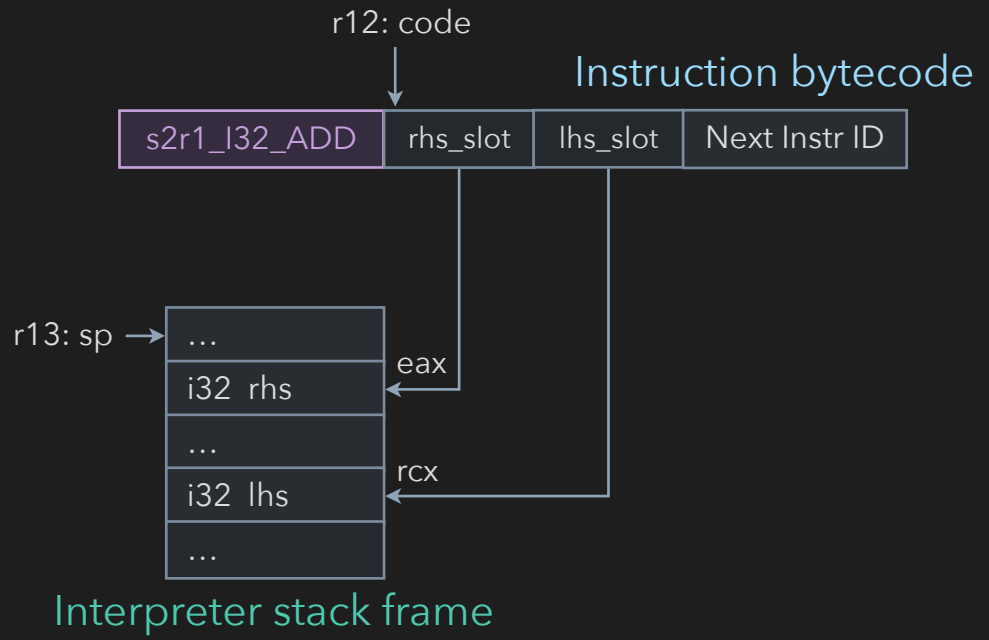
s2r1_I32Add

```
; Decode operand-slot offsets
movzx eax, word ptr [r12]          ; rhs slot
movzx ecx, word ptr [r12+2]        ; lhs slot

; Compute the second cached value
mov   edx, dword ptr [r13+rcx*4]   ; lhs = frame[lhs_slot]
add   edx, dword ptr [r13+rax*4]   ; r1 = lhs + frame[rhs_slot]

; Decode the next handler and advance code
movzx eax, word ptr [r12+4]        ; next handler ID
add   r12, 6                       ; skip two slots and the ID

; Dispatch using the cached table address
and   eax, 0FFFh                   ; mask the index
mov   rax, qword ptr [rsi+rax*8]   ; next = table[index]
jmp   rax                          ; tail-jum
```



Instruction handler optimization

s2s_I32Add

```
;Read operand-slot offsets
movzx eax, word ptr [rcx]
movzx r10d, word ptr [rcx+2]

;Read from slots and compute result in r10
mov   r10d, dword ptr [rdx+r10*4]
add   r10d, dword ptr [rdx+rax*4]

;Decode dest. slot and store the result
Movzx eax, word ptr [rcx+4]
mov   dword ptr [rdx+rax*4], r10d

;Read next handler ID and advance code
movzx eax, word ptr [rcx+6]
add   rcx, 8

;Bound the index and tail-dispatch
and   eax, 0FFFh
lea   r10, [rip+kInstructionTable]
jmp   qword ptr [r10+rax*8]
```

s2r_I32Add

```
;Read operand-slot offsets
movzx eax, word ptr [rcx]
movzx r9d, word ptr [rcx+2]

;Read from slots and compute result in r0
mov   r9d, dword ptr [rdx+r9*4]
add   r9d, dword ptr [rdx+rax*4]

;Read next handler ID and advance code
movzx eax, word ptr [rcx+4]
add   rcx, 6

;Bound the index and tail-dispatch
and   eax, 0FFFh
lea   r10, [rip+kInstructionTable]
jmp   qword ptr [r10+rax*8]
```

s2r1_I32Add

```
; Decode operand-slot offsets
movzx eax, word ptr [r12]
movzx ecx, word ptr [r12+2]

; Compute the second cached value
mov   edx, dword ptr [r13+rcx*4]
add   edx, dword ptr [r13+rax*4]

; Decode the next handler and advance code
movzx eax, word ptr [r12+4]
add   r12, 6

; Dispatch using the cached table address
and   eax, 0FFFh
mov   rax, qword ptr [rsi+rax*8]
jmp   rax
```

Performance analysis

preserve_none prototype vs. baseline interpreter:

- ~**5.2%** speed-up on x64 (same benchmarks and test configuration)
- ~**5.0%** speed-up on ARM64 (M4 Pro), despite the larger general-purpose register set

Further significant gains proved surprisingly difficult to obtain, even after extensive AI-assisted optimization experiments

- Dispatch remains: decoding, table lookup, indirect branch for each executed interpreter instruction
- Prediction matters: predictable transitions are cheaper than mispredictions
- Registers remain finite: complex handlers can still spill

The cost of dispach

```
movzx eax, [r12+4]     ; decode next opcode ID
and   eax, 0FFFh       ; bound to table
mov   rax, [rsi+rax*8] ; next = table[index]
jmp   rax              ; dispatch
```

The dispatch cost depends on branch prediction:

Predicted:	the front end runs ahead	~2-4 CPU cycles
Mispredicted:	pipeline flush + resolve chain	~20-30 CPU cycles

So we design for the predictor:

- musttail: every handler gets its own dispatch site, so the predictor learns opcode→successor (bytecode is highly biased)
- noinline: stops tail-merging/ICF from collapsing those sites back into one
- DISABLE_CFI_ICALL: No CFI check on the critical chain

Why we kept the handlers in C++

Clang produces efficient code for instruction handlers

- Tail-call and calling-convention contracts give us explicit control over their interfaces

Familiar tooling

- Inspect and debug the handlers as C++ functions
- Use sanitizers to investigate implementation errors

Less engine-specific machinery

- Express instruction semantics directly in C++, without learning a separate code-generation API with its own IR

Q&A